

Smart Recruitment Assistant

Predicting Candidate Job-Change Intent from HR Enrollment Data

Team 2: Project Report

1. Problem Statement

A data science training provider runs courses for a large pool of candidates, but only some of them are looking to change jobs once they finish. The rest take the course for other reasons: upskilling in their current role, general interest, or requirements from their employer. Right now, the company has no reliable way to separate these two groups, so its recruiting team ends up spending time and budget on candidates who were never going to move, while candidates who genuinely wanted a new job can slip through unnoticed.

The goal of this project is to build a model that predicts, for each enrolled candidate, the probability that they are looking for a job change. This score can then be used to prioritize outreach, focus recruiting effort where it is most likely to pay off, and give the company a data-driven way to plan training and hiring costs instead of treating every candidate the same.

2. Dataset Description

The data is HR analytics data from a Big Data / Data Science training program, split into a labeled training set and an unlabeled test set.

- `aug_train.csv`: 19,158 candidates, 14 columns, including the target label.
- `aug_test.csv`: 2,129 candidates, the same 13 feature columns but no target, since these are the candidates, the model needs to score.

Each row represents one candidate, with fields covering their city and its development index, gender, whether they have relevant data science experience, university enrollment status, education level, major, years of experience, current company size and type, years since their last job change, and hours of training completed.

The target is imbalanced: about 75% of candidates (14,381) are not looking for a new job, and about 25% (4,777) are. Several columns also have a substantial amount of missing data, most notably `company_type` (32%), `company_size` (31%), `gender` (23.5%), and `major_discipline` (14.7%). This mattered for the modeling approach in two ways: models like SMOTE and most classifiers can't handle missing values directly, and the missingness itself carried useful signal. Candidates with no listed `company_size` or `company_type` turned out to have a noticeably higher rate of looking for a new job than candidates with a known value, which is why this was captured as its own feature rather than just imputed away.

3. Methodology

The full workflow, from cleaning to the final model, was built as a single scikit-learn pipeline. The point of doing it this way is that the exact steps fit on the training data can be reapplied, unchanged, to any new test file, instead of writing separate preprocessing code for train and test.

Cleaning. The `experience` and `last_new_job` columns arrive as text categories ('>20', '<1', 'never', '>4'), so these were converted to numeric values before anything else ('>20' to 21, '<1' to 0, 'never' to 0, '>4' to 5).

Feature engineering. Three features were added on top of the raw columns: `no_company_info`, a flag for candidates missing both `company_size` and `company_type`; `job_change_rate`, `last_new_job` divided by (`experience` + 1); and `training_intensity`, `training_hours` divided by (`experience` + 1). These were meant to capture stability patterns that the raw columns don't show on their own.

Encoding and imputation. Numeric features (`city_development_index`, `training_hours`, `experience`, `last_new_job`, plus the three engineered features) were median-imputed and standardized. `education_level` and `company_size` were treated as ordinal and encoded with a defined order, with missing values filled as an explicit "Unknown" category rather than a middle value. The remaining categorical columns (`gender`, `relevant_experience`, `enrolled_university`,

major_discipline, company_type) were one-hot encoded, also filling missing values as "Unknown". The city column was dropped since every city map to exactly one city_development_index value, so keeping both would be redundant, and enrollee_id was dropped as it's just an identifier. After encoding, the feature set has 33 columns.

Handling class imbalance. SMOTE was used inside the pipeline to oversample the minority class, applied only during training and never to the validation or test data. Earlier in development, class_weight='balanced' (or scale_pos_weight for XGBoost) was also applied on top of SMOTE, but that meant the imbalance was being corrected twice at once, which pushed recall for the minority class up artificially and hurt precision. The final pipeline uses SMOTE as the only imbalance-handling step.

Model comparison. Seven classifiers were trained and compared on an 80/20 stratified train/validation split, using ROC-AUC as the main metric since it doesn't depend on picking a threshold: Logistic Regression, Decision Tree, Random Forest, KNN, SVM, XGBoost, and LightGBM.

Hyperparameter tuning. The four strongest models from that comparison, Random Forest, LightGBM, XGBoost, and SVM, were tuned further with RandomizedSearchCV using 3-fold cross-validation, scored on ROC-AUC.

Final model and deployment. Once the best tuned model was selected, it was refit on the complete training set (all 19,158 rows, not just the 80% split used earlier for comparison) and then applied to aug_test.csv to generate predictions. The fitted pipeline was saved with joblib so it can be reused directly on any future batch of candidates without rewriting any preprocessing code.

4. Results

Model comparison on the held-out validation set (ROC-AUC), before tuning:

Model	ROC-AUC
LightGBM	0.813
XGBoost	0.812
SVM	0.803
Random Forest	0.795
Logistic Regression	0.790
KNN	0.729
Decision Tree	0.639

After hyperparameter tuning, LightGBM stayed the strongest model:

Model	Validation AUC (before)	Validation AUC (after tuning)
LightGBM	0.813	0.814
XGBoost	0.812	0.812
Random Forest	0.795	0.806
SVM	0.803	0.803

The tuned LightGBM model was selected as the final model. At the default 0.5 probability threshold, its classification report on the validation set (3,832 candidates) was:

Class	Precision	Recall	F1-score	Support
Not looking (0)	0.91	0.80	0.85	2,877
Looking (1)	0.55	0.76	0.64	955

Overall accuracy was 79%. The model correctly identifies 76% of candidates who are actually looking for a new job (recall for class 1), at the cost of a fair number of false positives (precision of 0.55 for that class). A precision-recall curve was generated for the final model so the classification threshold can be adjusted depending on whether the

business wants to catch truer job-seekers or reduce false alarms; 0.5 is a reasonable starting point but not necessarily the best one for every use case.

Looking at what drives the predictions, `city_development_index` was the strongest single signal: candidates from less-developed cities are far more likely to be job-hunting than those from well-developed ones. `relevant_experience` and `education_level` also mattered, candidates without relevant data science experience, or with lower education levels, showed clearly higher job-change rates than the rest.

5. Conclusions

The tuned LightGBM model reaches a validation ROC-AUC of about 0.81, which is a solid result for this kind of noisy, partly-missing HR data, and clearly separates it from a naive baseline (predicting everyone as "not looking" would give 75% accuracy but zero recall on the group the company actually cares about). At the default threshold it correctly flags roughly three out of four candidates who are genuinely looking for a new job, which is the group the recruiting team most wants to reach.

From a business standpoint, this model can be used to rank incoming candidates by their probability of looking for a job change, so the recruiting team can prioritize outreach toward the candidates most likely to convert, instead of treating every enrollee the same way. It can also help with planning: if the company expects a certain fraction of a cohort to be genuinely job-hunting, it can budget recruiting time and resources accordingly rather than guessing.

There are some limitations worth keeping in mind. The dataset has a lot of missing data in exactly the columns that turned out to matter most (`company_size`, `company_type`), so the model is partly leaning on missingness patterns that may not generalize well if the company's data collection process changes. The precision on the minority class (0.55) also means close to half of the candidates flagged as "looking" are false positives, so this model is better used to prioritize and rank candidates than to make a hard yes/no decision on its own.

For future work, it would be worth testing feature importance more rigorously (e.g. SHAP values) to better explain individual predictions to the recruiting team, tuning the decision threshold against actual business costs instead of the default 0.5, and revisiting the model periodically as new cohorts of candidates come in, since the mix of cities and company types in the data will likely shift over time.